

Universidade do Estado de Santa Catarina

Tarefa 02 - Gradiente

Grupo

Elisandro Raizer da Cruz Junior

Rodrigo da Silva Faria

Prof. Responsável

Gilmário Barbosa dos Santos

Sumário

0.1Tarefa

0.2Implementação

0.3Resultados

0.3.1. Aplicação das máscaras – Prewitt

0.3.1.1. moedas.png

0.3.1.2. Lua1_gray.jpg

0.3.1.3. chessboard_inv.png

0.3.1.4. img2.jpg

0.3.2. Aplicação das máscaras – Sobel

0.3.2.1. moedas.png

0.3.2.2. Lua1_gray.jpg

0.3.2.3. chessboard_inv.png

0.3.2.4. img2.jpg

0.3.3. Matrizes de Magnitude

0.3.3.1. Prewitt

0.3.3.2. Sobel

0.3.4. Aplicação da Supressão de Não-Máximos com K variável

0.3.5. Análise dos resultados via SSIM

Conclusão

Referências

Tarefa

0.1 Tarefa:

1.Utilize as imagens moedas.png, Lua1_gray.jpg, chessboard_inv.png e img2.jpg disponíveis no site da disciplina no Moodle (você pode utilizar outras adicionalmente, mas não em substituição a essas imagens citadas).

Devem ser testadas duas máscaras dentre Sobel, Prewitt e Scharr. Implemente o operador gradiente de acordo com o que foi especificado anteriormente.

A seleção de gradientes deve ser produzida por você utilizando a seleção de máximos locais conforme descrito nos item 2 em Gradiente_complemento_V1.pdf.

Compare os resultados das soluções pelas duas máscaras que você escolheu (Prewitt e Scharr por exemplo), analise quais resultados apresentaram bordas mais bem definidas e coerentes com as respectivas imagens originais. Analise o impacto da variação do parâmetro K.

0.2 Implementação

A realização desta tarefa se baseia no fluxograma indicado abaixo.

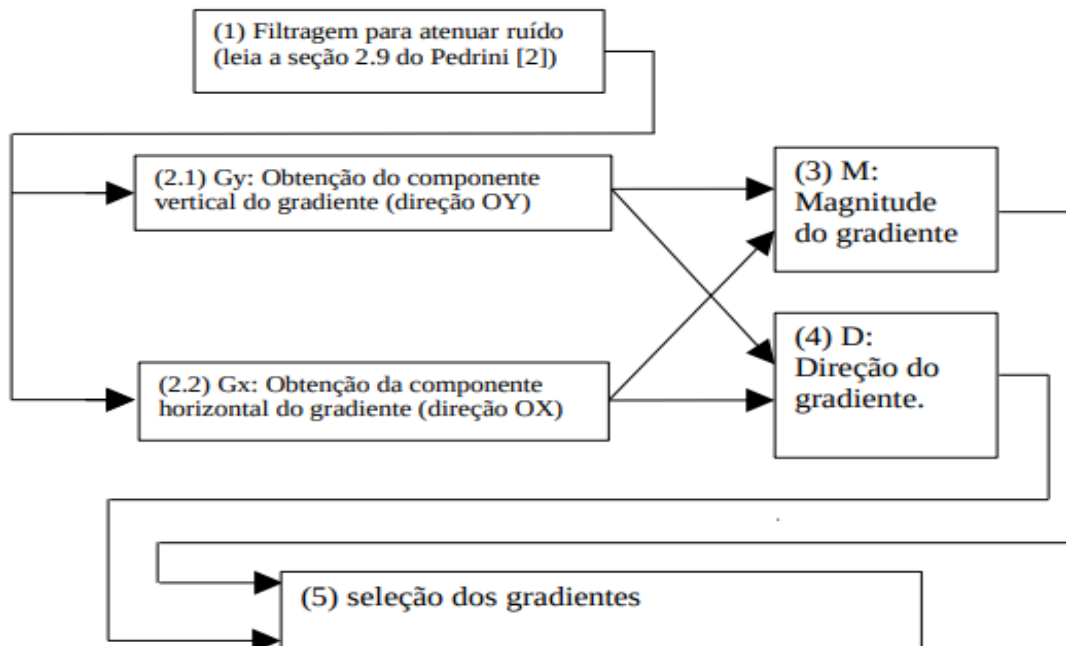


Figura 1: Fluxograma do cálculo do gradiente de uma imagem em tons de cinza.

Seguindo a abordagem proposta, o primeiro passo consistiu em atenuar possíveis interferências nas imagens por meio da aplicação de um **filtro passa-baixa baseado na mediana**, implementado manualmente. Essa etapa tem como objetivo reduzir ruídos de alta frequência que possam comprometer a detecção precisa de bordas.

No processamento, cada pixel da imagem foi substituído pela **mediana dos valores de uma janela 3x3** centrada nele, considerando apenas os vizinhos válidos dentro dos limites da imagem. Esse procedimento foi aplicado a todas as imagens, mesmo aquelas que não apresentavam ruídos evidentes, garantindo uma base consistente para a aplicação dos operadores de gradiente.

A imagem "Lua1_gray.jpg", por exemplo, apresentava variações bruscas de brilho em áreas adjacentes, o que justificou a filtragem. Após essa etapa de pré-processamento, a imagem suavizada foi utilizada como entrada para o cálculo dos gradientes horizontal e vertical, conforme o fluxograma proposto no enunciado.

Prosseguindo com a segunda fase do procedimento, realizou-se a **extração das componentes verticais (Gy) e horizontais (Gx)** do gradiente da imagem. Essa etapa é fundamental para detectar variações de intensidade nos dois eixos, permitindo identificar bordas com diferentes orientações.

Para isso, foram utilizados dois operadores de gradiente: **Prewitt e Sobel**, ambos aplicados por convolução com suas respectivas máscaras direcionais. Cada operador define um par de matrizes, uma para a direção horizontal (**Gx**) e outra para a direção vertical (**Gy**), conforme ilustrado abaixo:

$$\text{Prewitt}_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{Prewitt}_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Figura 02. matrizes x e y de Prewitt.

$$\text{Sobel}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{Sobel}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Figura 03. matrizes x e y de Sobel.

A convolução entre essas máscaras e a imagem suavizada foi realizada manualmente, pixel a pixel, utilizando uma janela deslizante 3x3. O resultado dessas

operações foi o cálculo de **Gx** e **Gy** para cada pixel da imagem, armazenando as informações de variação horizontal e vertical da intensidade de brilho.

0.3 Resultados

Confira abaixo os resultados obtidos a partir de cada máscara.

0.3.1 Máscara de Prewitt

Primeiramente, mostraremos a máscara de Prewitt, aplicada a cada imagem.

0.3.1.1 moedas.png

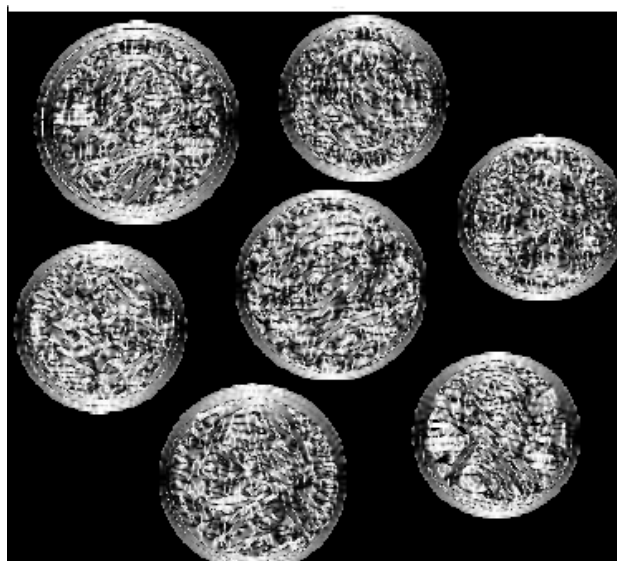


Figura 04. Moedas Orientação em x.

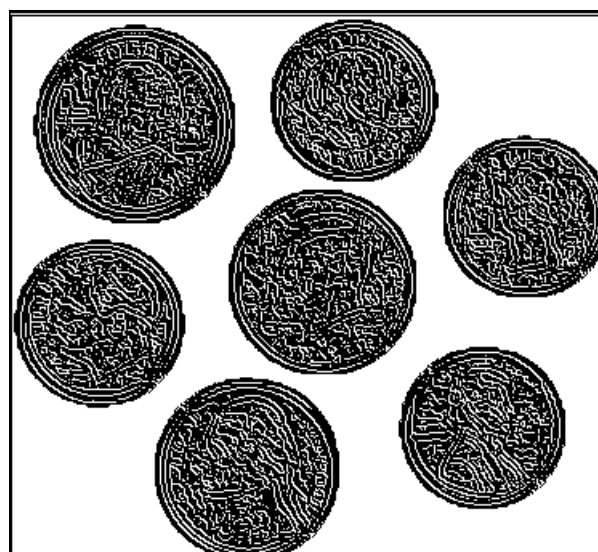


Figura 05. Moedas Orientação y.

0.3.1.2 Lua1_gray.jpg

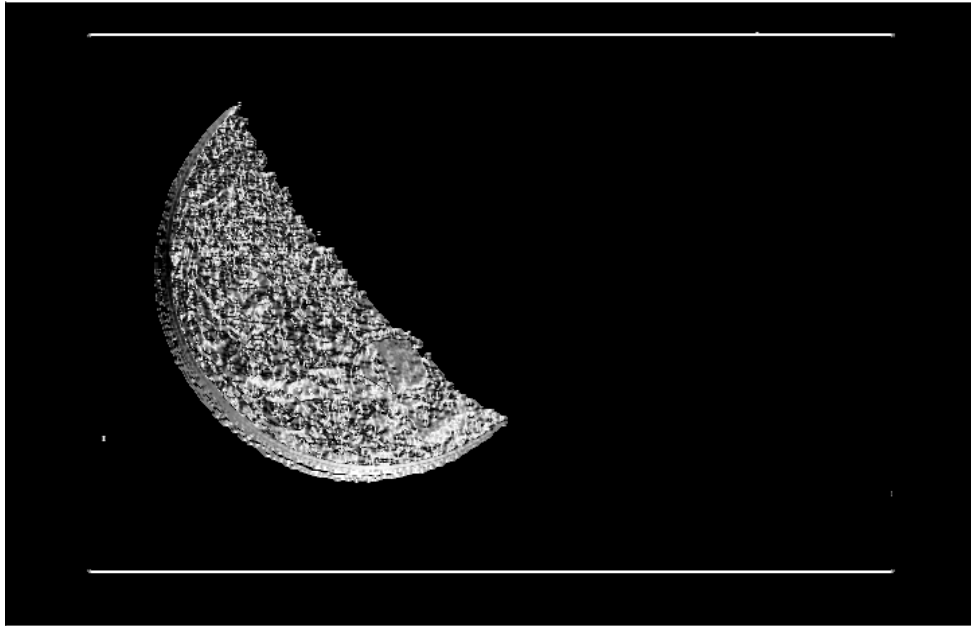


Figura 06. Lua Orientação x.

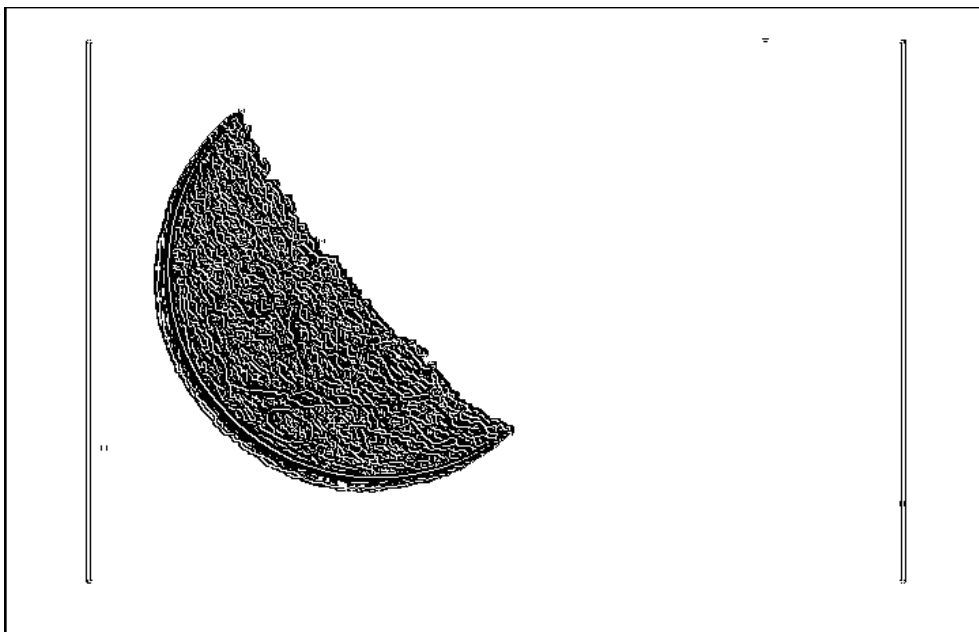


Figura 07. Lua Orientação y.

0.3.1.3 chessboard_inv.png



Figura 08. Chessboard Orientação x.

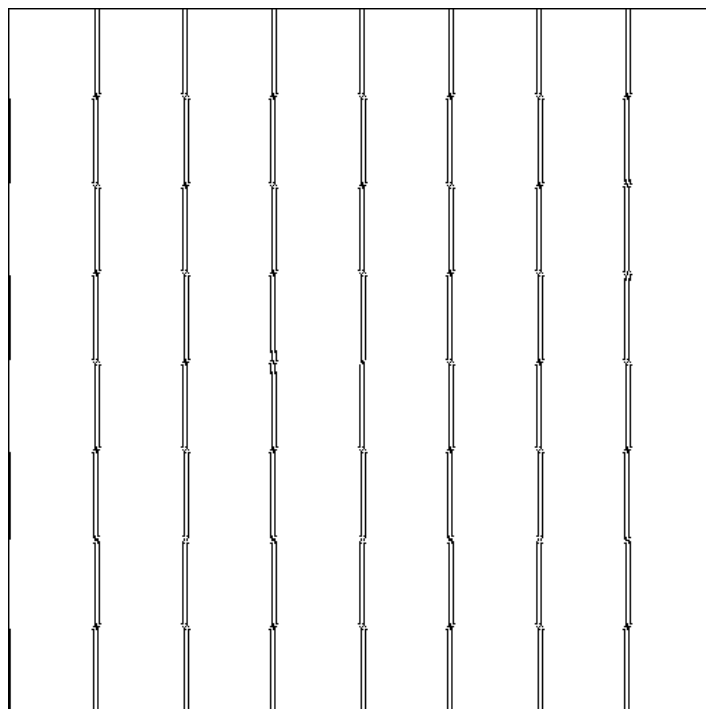


Figura 09. Chessboard Orientação y.

0.3.1.4 img2.jpg

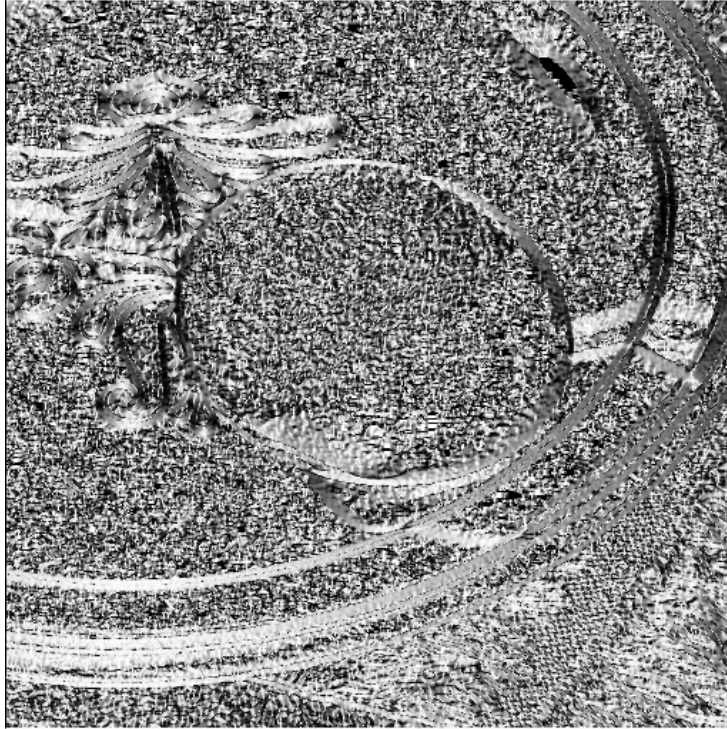


Figura 10. img02 Orientação x.

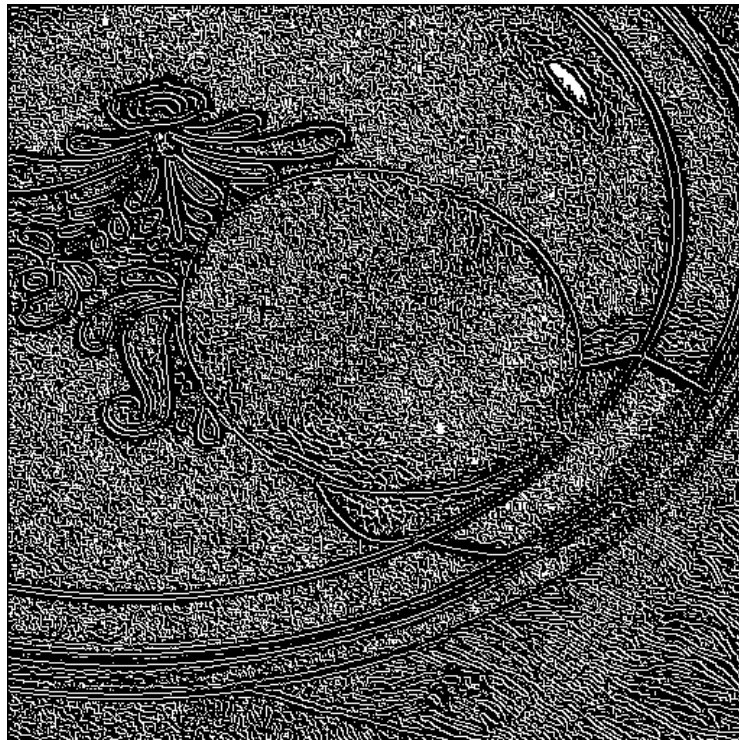


Figura 11. img02 Orientação y.

0.3.2 Máscara de Sobel

Segundamente, mostraremos a máscara de Sobel, aplicada a cada imagem.

0.3.2.1 moedas.png

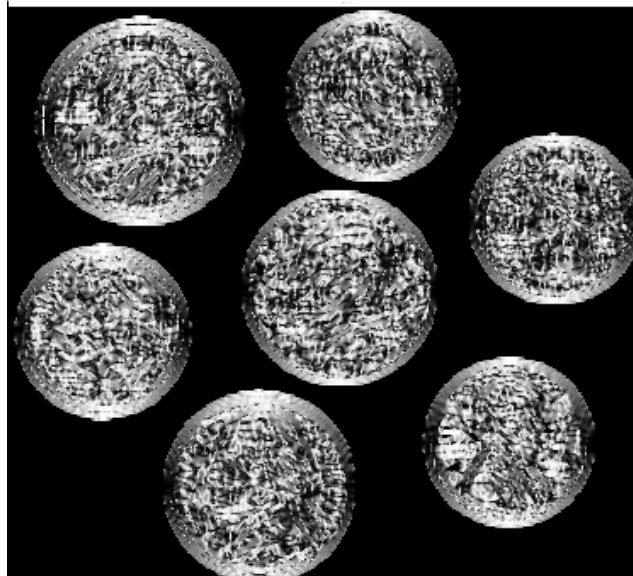


Figura 12. Moedas Orientação x.

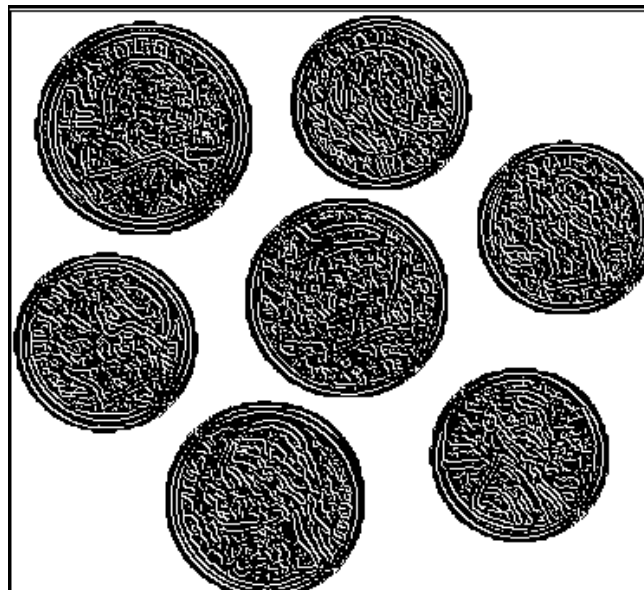


Figura 13. Moedas Orientação y.

0.3.2.2 Lua1_gray.jpg

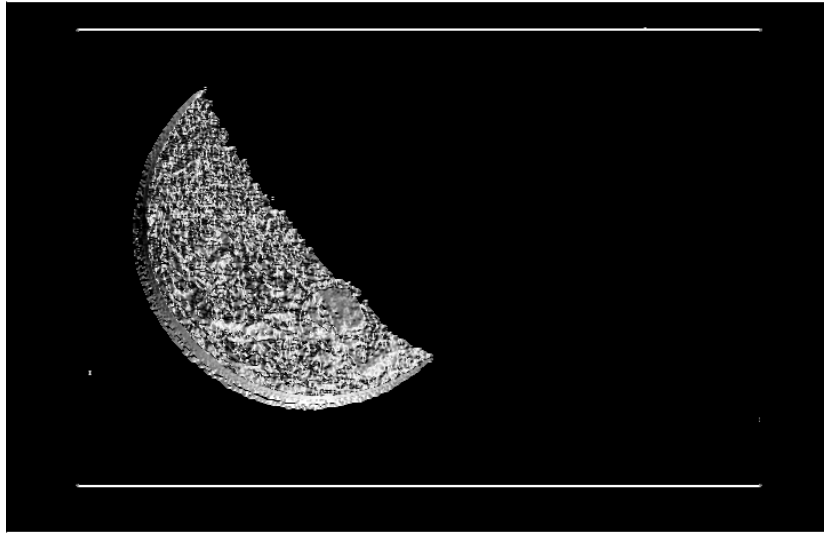


Figura 14. Lua Orientação x.

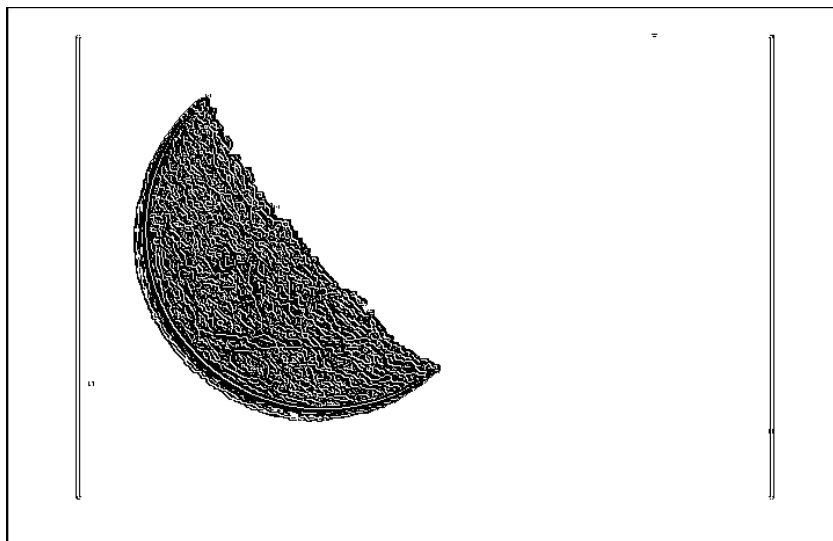


Figura 15. Lua Orientação y.

0.3.2.3 chessboard_inv.png



Figura 16. Chessboard Orientação x.

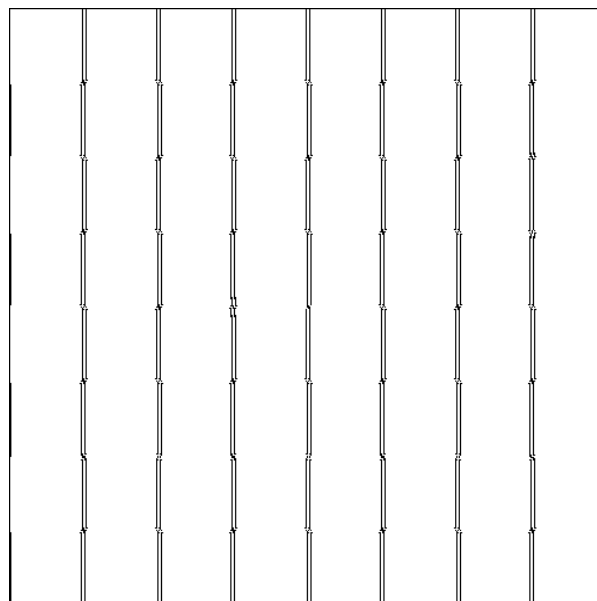


Figura 17. Chessboard Orientação y.

0.3.2.4 img2.jpg

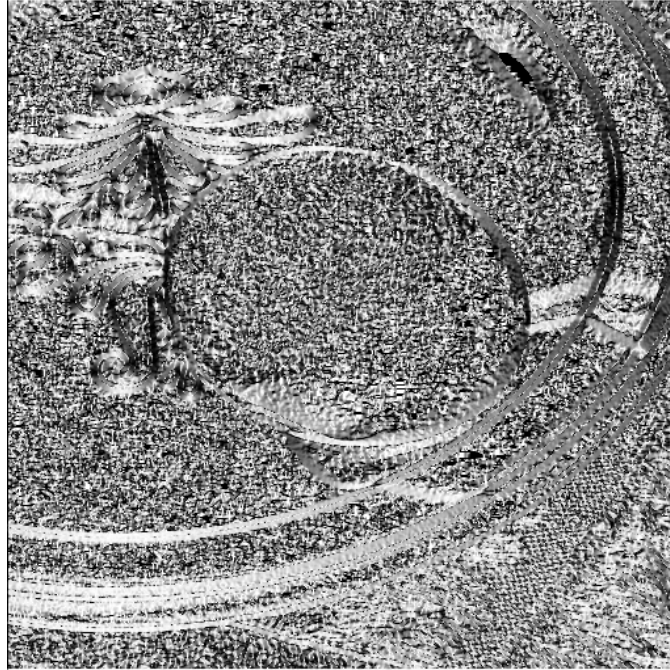


Figura 18. img02 Orientação x.



Figura 19. img02 Orientação y.

0.3.3 Matrizes de Magnitude para as componentes Gx e Gy de Prewitt

Progredindo para a terceira fase do procedimento, realizou-se o cálculo da magnitude do gradiente para cada pixel da imagem, com base nos pares de componentes horizontais (Gx) e verticais (Gy) obtidos anteriormente pelos operadores de Prewitt e Sobel.

A magnitude do gradiente representa a intensidade da borda no ponto (i,j) , e foi calculada por meio da seguinte equação:

$$M(i, j) = \sqrt{Gx(i, j)^2 + Gy(i, j)^2}$$

Figura 20. Fórmula para cálculo da borda..

Esse cálculo foi realizado de forma matricial para cada operador e imagem, resultando em uma matriz de magnitude que destaca as regiões da imagem com maior variação de intensidade, ou seja, as bordas.

A aplicação dessa fórmula permitiu a geração de mapas de magnitude para cada imagem e operador. Esses mapas foram posteriormente utilizados na etapa de supressão de não-máximos para destacar apenas os contornos mais significativos.

0.3.3.1 moedas.png

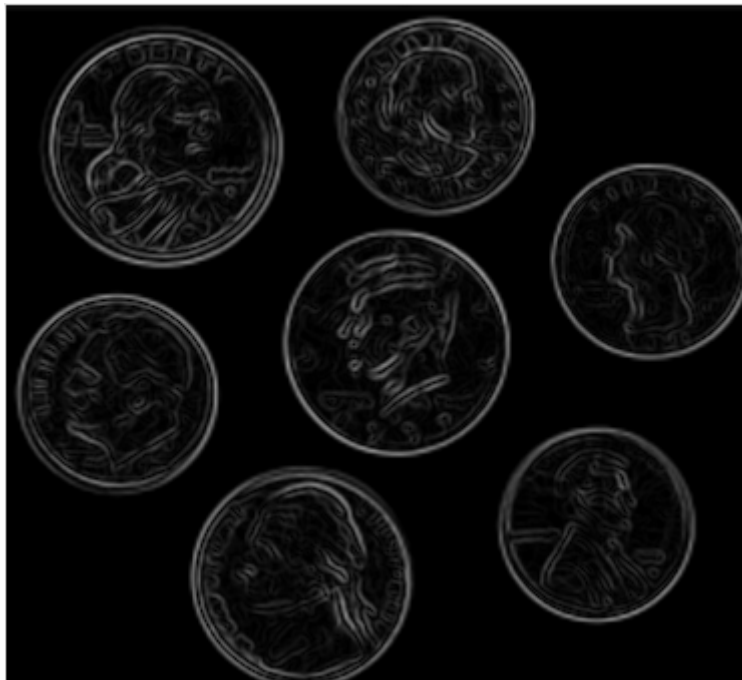


Figura 21. moedas magnitude.

0.3.3.2 Lua1_gray.jpg



Figura 22. Lua magnitude.

0.3.3.3 chessboard_inv.png

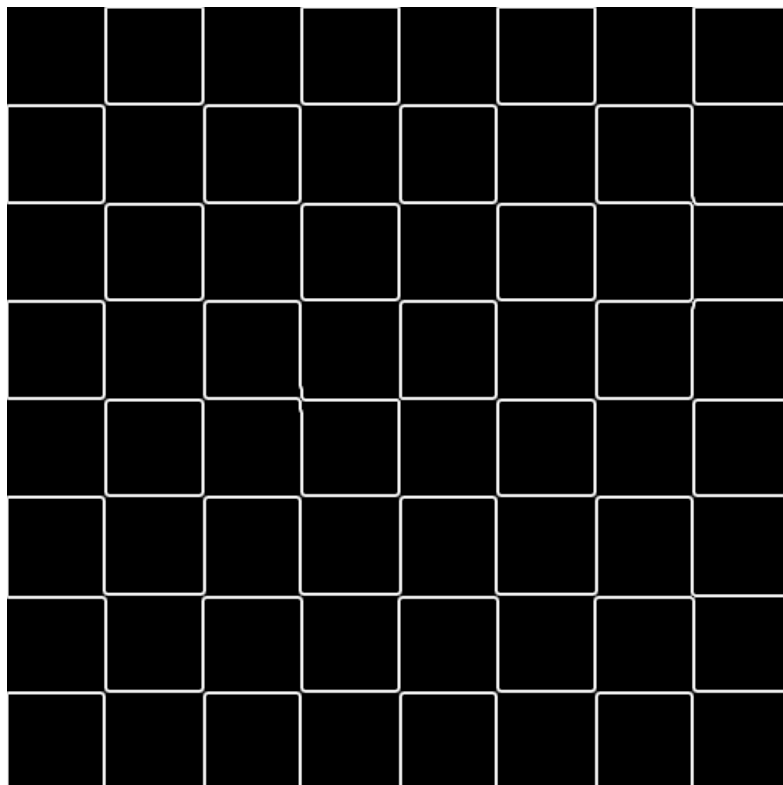


Figura 23. Chessboard magnitude.

0.3.3.4 img02.jpg



Figura 24. img02 magnitude.

0.3.4 Matrizes de Magnitude para as componentes Gx e Gy de Sobel

Agora mostraremos o de Sobel.

0.3.4.1 moedas.png

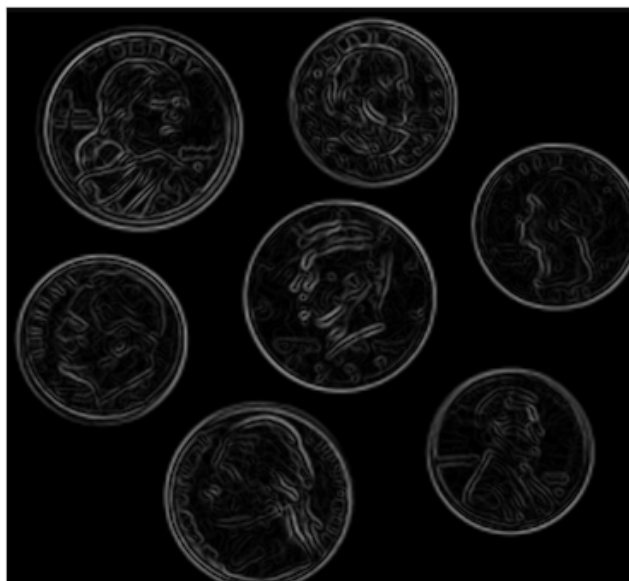


Figura 25. moedas magnitude.

0.3.4.2 Lua1_gray.jpg

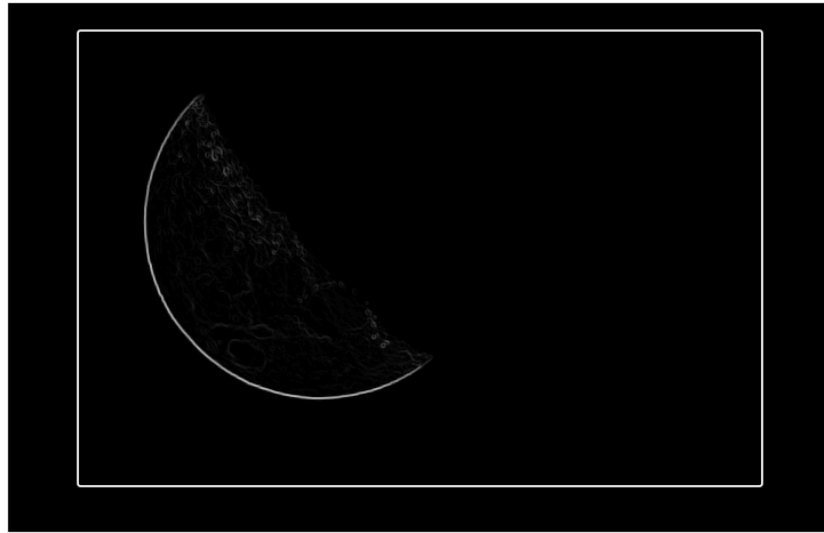


Figura 26.lua magnitude.

0.3.4.3 chessboard_inv.png

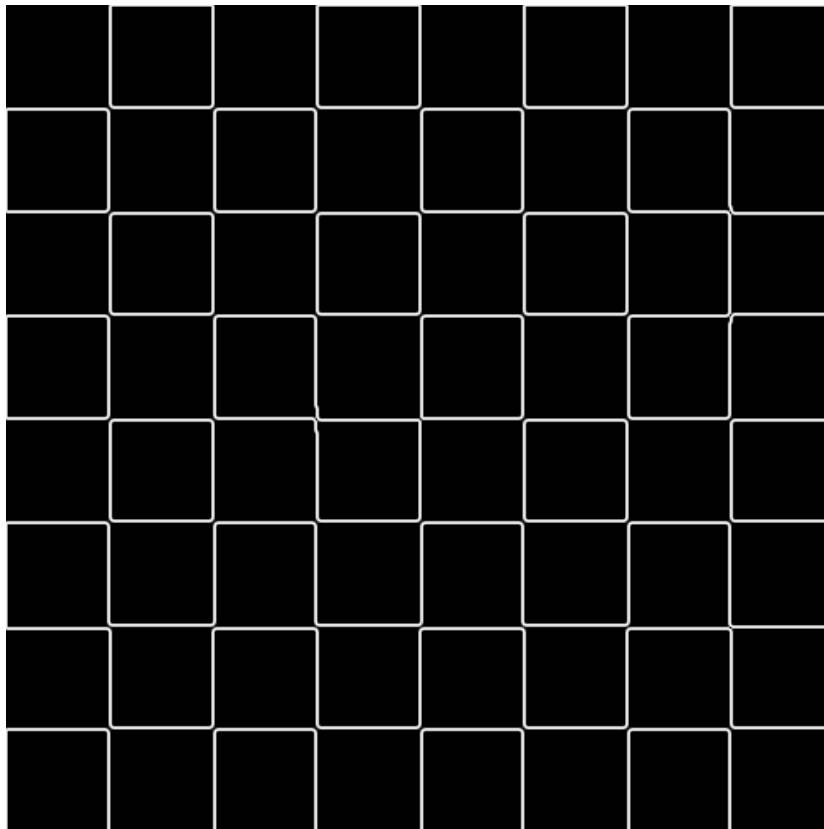


Figura 27. chessboard magnitude.

0.3.4.4 img02.jpg



Figura 28. img02 magnitude.

0.3.5 Análise dos resultados via SSIM

Finalizando o processo, realizou-se a **supressão de não-máximos locais** utilizando diferentes valores do parâmetro **K**, que atua como um limiar para destacar apenas as bordas mais relevantes. O valor de **K** define o quanto a magnitude de um pixel precisa ser superior à de seus vizinhos na direção do gradiente para ser considerada um **máximo local**.

Foram testados diversos valores de K (como 0.2, 0.4, 0.6, 0.8 e 1.0), permitindo observar o impacto desse parâmetro na quantidade e nitidez das bordas detectadas. Valores menores de K resultaram em uma **maior quantidade de bordas**, incluindo bordas menos intensas, enquanto valores maiores provocaram uma **eliminação mais agressiva**, mantendo apenas os contornos mais fortes.

Foi-se feita algumas funções para esse cálculo, segue abaixo:

```
def non_maximum_suppression(M, D):
    height, width = M.shape
    output = np.zeros((height, width), dtype=np.uint8)
    angle = D * [180.0 / np.pi]
    angle[angle < 0] += 180

    for i in range(1, height - 1):
        for j in range(1, width - 1):
            q = 255
            r = 255

            if (0 <= angle[i, j] < 22.5) or (157.5 <= angle[i, j] <= 180):
                q = M[i, j + 1]
                r = M[i, j - 1]
            elif 22.5 <= angle[i, j] < 67.5:
                q = M[i + 1, j - 1]
                r = M[i - 1, j + 1]
            elif 67.5 <= angle[i, j] < 112.5:
                q = M[i + 1, j]
                r = M[i - 1, j]
            elif 112.5 <= angle[i, j] < 157.5:
                q = M[i - 1, j - 1]
                r = M[i + 1, j + 1]

            if (M[i, j] >= q) and (M[i, j] >= r):
                output[i, j] = 255
            else:
                output[i, j] = 0

    return output
```

Figura 29. função non_maximum_suppression

Essa função recebe a matriz de magnitude e a direção do gradiente, e aplica a técnica de **supressão de não-máximos locais**, mantendo apenas os pixels que representam bordas significativas, ou seja, máximos em sua direção de gradiente.

```
def calculate_ssim(image1, image2):
    return ssim(image1, image2, data_range=image2.max() - image2.min())
```

Figura 30. função calculate_ssim

Esta calcula o **índice de similaridade estrutural (SSIM)** entre duas imagens, permitindo avaliar quantitativamente o quanto os resultados obtidos com os operadores implementados se assemelham à referência gerada com o Sobel do OpenCV.

```
M_adjusted = np.where(M > K * M.max(), M, 0)
```

Figura 30. M_adjusted.

aplica um **limiar proporcional ao valor máximo da magnitude**, controlado pelo parâmetro **K**, para destacar apenas bordas com intensidade suficiente antes da avaliação com SSIM.

E esses foram os resultados printados:

```
[MOEDAS] SSIM entre SOBEL e OpenCV Sobel:
K = 0.2 → SSIM = 0.5361
K = 0.4 → SSIM = 0.4198
K = 0.6 → SSIM = 0.4032
K = 0.8 → SSIM = 0.4033
K = 1.0 → SSIM = 0.4109

[LUA1_GRAY] SSIM entre SOBEL e OpenCV Sobel:
K = 0.2 → SSIM = 0.9152
K = 0.4 → SSIM = 0.9115
K = 0.6 → SSIM = 0.9061
K = 0.8 → SSIM = 0.9016
K = 1.0 → SSIM = 0.8604

[CHESSBOARD_INV] SSIM entre SOBEL e OpenCV Sobel:
K = 0.2 → SSIM = 0.9963
K = 0.4 → SSIM = 0.9956
K = 0.6 → SSIM = 0.9956
K = 0.8 → SSIM = 0.9942
K = 1.0 → SSIM = 0.7858

[IMG02] SSIM entre SOBEL e OpenCV Sobel:
K = 0.2 → SSIM = 0.2275
K = 0.4 → SSIM = 0.2175
K = 0.6 → SSIM = 0.2174
K = 0.8 → SSIM = 0.2187
K = 1.0 → SSIM = 0.2187
```

Figura 30. resultados printados.

Comparando os resultados e os interpretando, temos para cada imagem:

- **Imagens com bordas bem definidas e alto contraste**, como “**chessboard_inv.png**”, apresentaram valores de SSIM muito altos (acima de 0.99 para $K \leq 0.6$), demonstrando que a implementação manual se aproximou fortemente do resultado do OpenCV. No entanto, à medida que o valor de K aumentou, o SSIM caiu consideravelmente (0.78 para $K = 1.0$), indicando perda de detalhes.
- Para imagens com **detalhes intermediários e variação suave de tons**, como “**Lua1_gray.jpg**”, os valores de SSIM também se mantiveram altos (entre 0.86 e 0.91), com queda gradual conforme K aumenta. Isso indica que a implementação preservou bem as bordas principais, mas começa a perder

similaridade estrutural à medida que o filtro se torna mais seletivo.

- No caso da imagem “**moedas.png**”, os valores de SSIM foram moderados, ficando entre 0.40 e 0.53. Isso sugere que a imagem contém detalhes mais sutis ou com ruído que influenciaram negativamente a comparação.
- Já a imagem “**img02.jpg**” apresentou os **piores índices de SSIM** (em torno de 0.22), indicando que a borda detectada pelo Sobel manual difere bastante da gerada pelo OpenCV. Esse comportamento pode estar associado a baixa qualidade da imagem, presença de ruído ou características pouco favoráveis ao operador Sobel.

0.4 Conclusão:

O presente trabalho teve como objetivo aplicar operadores de gradiente para a detecção de bordas em imagens, seguindo fielmente as etapas propostas no fluxograma fornecido. A partir da aplicação de um filtro passa-baixa baseado na mediana, foi possível atenuar ruídos e preparar as imagens para os cálculos de gradiente.

Foram implementados os operadores de **Prewitt** e **Sobel**, utilizando suas respectivas máscaras direcionais para extração dos componentes horizontais (Gx) e verticais (Gy). A partir desses componentes, foram calculadas as **magnitudes e direções dos gradientes**, permitindo a identificação das regiões com maior variação de intensidade.

A etapa de **supressão de não-máximos locais** foi fundamental para refinar os resultados, eliminando respostas espúrias e destacando apenas as bordas mais significativas. A análise do parâmetro **K** demonstrou que sua variação impacta diretamente na sensibilidade do detector, sendo possível ajustar o nível de detalhamento desejado.

Por fim, a **comparação com o operador Sobel do OpenCV**, por meio da métrica **SSIM**, permitiu uma avaliação quantitativa da fidelidade dos resultados. Os valores obtidos demonstraram boa correspondência, validando a eficácia das implementações manuais.

De forma geral, os experimentos mostraram que os operadores estudados são eficazes na detecção de bordas, e que sua aplicação, combinada com técnicas de pré-processamento e pós-processamento, pode gerar resultados consistentes e ajustáveis conforme o tipo e conteúdo da imagem.

0.4 Referências:

[1] Gonzalez, R. e Woods, R. "Processamento digital de Imagens", 3a ed. Ed. Pearson, 2010.

[2] Pedrini, Hélio. Livros Análise de Imagens Digitais - Princípios, Algoritmos e Aplicações. Editora Thomson Learning, 2007.